

Module 7.4 - Chaining and Advanced Pipes

Learning Objectives: Master pipe chaining for complex data transformations, understand execution order and performance implications, and apply advanced pipe patterns for real-world scenarios.

1. Pipe Chaining Fundamentals

1.1 What is Pipe Chaining?

Pipe chaining allows you to apply multiple pipes in sequence to transform data through several operations. Each pipe receives the output of the previous pipe, creating a data transformation pipeline.

Basic Syntax:

```
{{ value | pipe1 | pipe2 | pipe3 }}
```

Execution Order:

1. The original value is passed to `pipe1`
2. The output of `pipe1` is passed to `pipe2`
3. The output of `pipe2` is passed to `pipe3`
4. The final result is displayed in the template

Key Concept: Pipes are executed from left to right, forming a transformation pipeline where each pipe processes the output of the previous one.

1.2 Simple Pipe Chaining Examples

Component TypeScript:

```
export class ChainingBasicsComponent {  
  productName: string = 'wireless bluetooth headphones';  
  price: number = 149.99;  
  discount: number = 0.25;  
  description: string = 'High-quality wireless headphones with active noise cancellation and  
30-hour battery life';  
}
```

Component Template:

```
<!-- Text transformation chaining -->  
<p>Product: {{ productName | titlecase | slice:0:20 }}</p>  
  
<!-- Number and currency chaining -->  
<p>Price: {{ price | number:'1.2-2' | currency:'USD':'symbol' }}</p>  
  
<!-- Text and slice chaining -->  
<p>Description: {{ description | uppercase | slice:0:50 }}</p>  
  
<!-- Discount display -->  
<p>Save: {{ discount | percent:'1.0-0' }}</p>
```

```
Product: Wireless Bluetooth  
Price: $149.99  
Description: HIGH-QUALITY WIRELESS HEADPHONES WITH ACTIVE NOI  
Save: 25%
```

2. Understanding Execution Order

2.1 Left-to-Right Processing

Pipes always execute from left to right. Understanding this order is crucial when chaining:

Example 1: Date → Uppercase

```
export class OrderComponent {  
  eventDate: Date = new Date(2026, 0, 29);  
}
```

```
<p>{{ eventDate | date:'fullDate' | uppercase }}</p>
```

THURSDAY, JANUARY 29, 2026

Execution Steps:

1. `eventDate` → `date` pipe → "Thursday, January 29, 2026"
2. "Thursday, January 29, 2026" → `uppercase` pipe → "THURSDAY, JANUARY 29, 2026"

Example 2: Titlecase → Slice

```
export class OrderComponent {  
  title: string = 'learn angular development step by step';  
}
```

```
<p>{{ title | titlecase | slice:0:25 }}</p>
```

Learn Angular Development

Execution Steps:

1. `title` → `titlecase` pipe → "Learn Angular Development Step By Step"
2. "Learn Angular Development Step By Step" → `slice:0:25` pipe → "Learn Angular Development"

2.2 Order Matters

The order in which you chain pipes can produce different results:

Component TypeScript:

```
export class OrderMattersComponent {  
  text: string = 'angular is a powerful framework';  
}
```

Template - Different Orders:

```
<!-- Order 1: Slice then Uppercase -->  
<p>Slice → Uppercase: {{ text | slice:0:10 | uppercase }}</p>  
  
<!-- Order 2: Uppercase then Slice -->  
<p>Uppercase → Slice: {{ text | uppercase | slice:0:10 }}</p>
```

```
Slice → Uppercase: ANGULAR IS  
Uppercase → Slice: ANGULAR IS
```

While this example produces similar results, in many cases the order significantly affects the output:

```
<!-- Different results with titlecase -->  
<p>Slice → Titlecase: {{ text | slice:0:20 | titlecase }}</p>  
<p>Titlecase → Slice: {{ text | titlecase | slice:0:20 }}</p>
```

```
Slice → Titlecase: Angular Is A Powerf  
Titlecase → Slice: Angular Is A Powerful
```

3. Advanced Chaining Patterns

3.1 Combining Text and Number Pipes

Component TypeScript:

```
export class AdvancedChainingComponent {  
  products = [  
    { name: 'laptop computer', price: 899.99, stock: 15 },
```

```
{ name: 'wireless mouse', price: 29.99, stock: 50 },
{ name: 'usb keyboard', price: 79.99, stock: 32 }
];
}
```

Component Template:

```
<div *ngFor="let product of products">
  <h3>{{ product.name | titlecase | slice:0:15 }}</h3>
  <p>Price: {{ product.price | currency:'USD':'symbol':'1.2-2' }}</p>
  <p>Stock: {{ product.stock | number:'3.0-0' }} units</p>
</div>
```

Laptop Computer
 Price: \$899.99
 Stock: 015 units

Wireless Mouse
 Price: \$29.99
 Stock: 050 units

Usb Keyboard
 Price: \$79.99
 Stock: 032 units

3.2 Chaining with Custom Pipes

You can chain custom pipes with built-in pipes:

Custom Pipes:

```
// highlight.pipe.ts
@Pipe({ name: 'highlight' })
export class HighlightPipe implements PipeTransform {
  transform(value: string, term: string): string {
    if (!term) return value;
    const regex = new RegExp(term, 'gi');
    return value.replace(regex, `${term}`);
  }
}
```

```

}

// excerpt.pipe.ts
@Pipe({ name: 'excerpt' })
export class ExcerptPipe implements PipeTransform {
  transform(value: string, length: number = 100): string {
    if (!value || value.length <= length) return value;
    return value.substring(0, length) + '...';
  }
}

```

Template:

```

export class SearchComponent {
  articles = [
    { title: 'Introduction to Angular', content: 'Angular is a powerful framework...' },
    { title: 'Angular Components', content: 'Components are the building blocks...' }
  ];
  searchTerm: string = 'Angular';
}

```

```

<div *ngFor="let article of articles">
  <h3>{{ article.title | highlight:searchTerm | titlecase }}</h3>
  <p [innerHTML]="article.content | excerpt:50 | highlight:searchTerm"></p>
</div>

```

4. Performance Considerations

4.1 Pipe Chain Length

Each pipe in a chain adds processing overhead. Follow these guidelines:

Chain Length	Recommendation	Performance Impact
1-2 pipes	✓ Ideal	Minimal
3-4 pipes	✓ Acceptable	Low

Chain Length	Recommendation	Performance Impact
5+ pipes	⚠ Consider refactoring	May impact performance

Best Practice: If you find yourself chaining 5 or more pipes, consider moving some of the transformation logic to a component method or getter.

4.2 Component Method Alternative

For complex transformations, consider using component methods:

Instead of this (too many pipes):

```
<p>{{ data | pipe1 | pipe2 | pipe3 | pipe4 | pipe5 | pipe6 }}</p>
```

Do this (component method):

```
// Component
export class MyComponent {
  data: string = 'some data';

  getTransformedData(): string {
    let result = this.data;
    // Apply transformations
    result = result.toUpperCase();
    result = result.substring(0, 20);
    // ... more transformations
    return result;
  }
}

// Template
<p>{{ getTransformedData() }}</p>
```

5. Practical Real-World Example

5.1 Product Dashboard

Let's build a comprehensive product dashboard using pipe chaining:

Component TypeScript:

```
export class ProductDashboardComponent {
  products = [
    {
      id: 1,
      name: 'wireless bluetooth headphones',
      category: 'electronics',
      price: 149.99,
      originalPrice: 199.99,
      soldUnits: 1243,
      revenue: 186387.57,
      rating: 4.6,
      maxRating: 5,
      growthRate: 0.18,
      launchDate: new Date(2025, 10, 20),
      lastSold: new Date(2026, 0, 29, 11, 42)
    },
    {
      id: 2,
      name: 'ergonomic office chair',
      category: 'furniture',
      price: 349.99,
      originalPrice: 349.99,
      soldUnits: 867,
      revenue: 303441.33,
      rating: 4.8,
      maxRating: 5,
      growthRate: 0.32,
      launchDate: new Date(2025, 8, 15),
      lastSold: new Date(2026, 0, 28, 15, 20)
    }
  ];

  reportDate: Date = new Date(2026, 0, 29, 9, 0, 0);
  totalRevenue: number = 2547892.50;

  getDiscount(original: number, current: number): number {
```



```

    return (original - current) / original;
  }
}

```

Component Template:

```

<div class="dashboard">
  <header>
    <h1>{{ 'product analytics dashboard' | titlecase }}</h1>
    <p>Report Date: {{ reportDate | date:'MMMM d, yyyy' }}</p>
    <p>Total Revenue: {{ totalRevenue | currency:'USD':'symbol':'1.2-2' }}</p>
  </header>

  <section class="products">
    <h2>{{ 'product performance' | uppercase }}</h2>

    <div class="product-card" *ngFor="let product of products">
      <!-- Product Name - Titlecase and truncate -->
      <h3>{{ product.name | titlecase | slice:0:30 }}</h3>

      <!-- Category Badge -->
      <span class="category">{{ product.category | uppercase }}</span>

      <!-- Price with discount -->
      <div class="pricing">
        <p>Current: {{ product.price | currency:'USD':'symbol':'1.2-2' }}</p>
        <p *ngIf="product.originalPrice > product.price">
          Original: {{ product.originalPrice | currency:'USD':'symbol':'1.2-2' }}
          (Save {{ getDiscount(product.originalPrice, product.price) | percent:'1.0-0' }})
        </p>
      </div>

      <!-- Sales metrics -->
      <div class="metrics">
        <p>Units Sold: {{ product.soldUnits | number:'1.0-0' }}</p>
        <p>Revenue: {{ product.revenue | currency:'USD':'symbol':'1.0-0' }}</p>
        <p>Rating: {{ product.rating | number:'1.1-1' }}/{{ product.maxRating }}</p>
        <p>Growth: {{ product.growthRate | percent:'1.1-1' }}</p>
      </div>

      <!-- Dates -->
      <div class="dates">
        <p>Launched: {{ product.launchDate | date:'MMM d, yyyy' }}</p>
        <p>Last Sold: {{ product.lastSold | date:'MMM d, h:mm a' }}</p>
      </div>
    </div>
  </section>
</div>

```

```
</div>
</section>

<footer>
  <p>Dashboard updated: {{ reportDate | date:'medium' }}</p>
</footer>
</div>
```

PRODUCT ANALYTICS DASHBOARD

Report Date: January 29, 2026

Total Revenue: \$2,547,892.50

PRODUCT PERFORMANCE

Wireless Bluetooth Headphones

ELECTRONICS

Current: \$149.99

Original: \$199.99 (Save 25%)

Units Sold: 1,243

Revenue: \$186,388

Rating: 4.6/5

Growth: 18.0%

Launched: Nov 20, 2025

Last Sold: Jan 29, 11:42 AM

Ergonomic Office Chair

FURNITURE

Current: \$349.99

Units Sold: 867

Revenue: \$303,441

Rating: 4.8/5

Growth: 32.0%

Launched: Sep 15, 2025

Last Sold: Jan 28, 3:20 PM

Dashboard updated: Jan 29, 2026, 9:00:00 AM

6. Common Chaining Patterns

6.1 Text Formatting Patterns

Pattern	Example	Use Case
Titlecase → Slice	<code>{{ text titlecase slice:0:20 }}</code>	Formatted preview text
Uppercase → Slice	<code>{{ text uppercase slice:0:10 }}</code>	Category tags/badges
Lowercase → Titlecase	<code>{{ email lowercase titlecase }}</code>	Clean user input

6.2 Number Formatting Patterns

Pattern	Example	Use Case
Number → Currency	<code>{{ price number:'1.2-2' currency }}</code>	Precise price display
Percent → Uppercase	<code>{{ rate percent uppercase }}</code>	Highlighted metrics
Number → Slice	<code>{{ large number slice:0:10 }}</code>	Abbreviated numbers

6.3 Date Formatting Patterns

Pattern	Example	Use Case
Date → Uppercase	<code>{{ date date:'short' uppercase }}</code>	Prominent date display
Date → Slice	<code>{{ date date:'full' slice:0:20 }}</code>	Abbreviated date text
Date → Titlecase	<code>{{ date date:'EEEE' titlecase }}</code>	Day name formatting

7. Best Practices

Pipe Chaining Best Practices:

- **Keep chains short:** Limit to 2-4 pipes for readability and performance
- **Order matters:** Think carefully about the sequence of transformations
- **Use component methods:** For complex logic, move to component code
- **Consider pure pipes:** Ensure all pipes in chain are pure for best performance
- **Document complex chains:** Add comments for non-obvious transformations
- **Test chains thoroughly:** Verify the output with different input values
- **Avoid impure pipes in chains:** Each impure pipe executes on every change detection

Common Pitfalls:

- ❌ Chaining too many pipes (5+ pipes)
- ❌ Using impure pipes in long chains
- ❌ Not considering execution order
- ❌ Complex transformations that belong in components
- ❌ Forgetting to handle null/undefined in chains

8. Practice Exercises

Exercise 1: Employee Badge Generator

Create an employee badge display that uses pipe chaining:

- Employee name in titlecase, sliced to 20 characters
- Department in uppercase
- Salary with currency and number formatting
- Hire date formatted as "MMM d, yyyy" then sliced

- Performance rating as percentage

Exercise 2: Blog Post Preview

Build a blog post preview using chained pipes:

- Title: titlecase → slice (first 40 chars)
- Author: uppercase
- Content: custom excerpt pipe → highlight search term
- Publish date: date → titlecase
- View count: number with thousands separator

Exercise 3: Product Comparison Table

Create a product comparison using multiple pipe chains:

- Product names with titlecase and truncation
- Prices with currency formatting
- Discount percentages
- Ratings with number precision
- Availability dates formatted consistently

Exercise 4: Sales Dashboard

Build a comprehensive sales dashboard that combines:

- Multiple text transformation chains
 - Currency and percentage formatting
 - Date formatting with different styles
 - Custom pipes with built-in pipes
 - At least 3 different chaining patterns
-

9. Summary

In this module, you learned:

- **Pipe Chaining:** Combining multiple pipes for complex data transformations
- **Execution Order:** Pipes execute left-to-right, order matters
- **Performance:** Keep chains short (2-4 pipes), avoid impure pipes in chains
- **Common Patterns:** Text, number, and date formatting combinations
- **Real-World Applications:** Building dashboards and complex displays
- **Best Practices:** Writing maintainable, performant pipe chains

Pipes are essential for clean, readable templates and efficient data transformation. Mastering pipe chaining allows you to create sophisticated displays while keeping your component logic focused on business rules.

Course Complete: You've now mastered Angular pipes! You understand fundamentals, built-in pipes, custom pipe creation, and advanced chaining techniques. Apply these skills to build professional, maintainable Angular applications.

Module 7.4 - Chaining and Advanced Pipes | Angular Framework Training